

Session 9.3 — Leakage & Validation

Learn to Avoid Common Traps in Model Training

Continuing from: Session 9.2 — Data Prep: Handling Messy Data

Duration: ~1.5 hours

Style: Teach first, practice later

Deliverables: This teaching guide (.md) + Leakage demo script (.py) + Cross-validation notebook (.ipynb)

What We've Covered So Far

Session	Topics
1.1–8.3	Python, Pandas, SQL, Visualization, EDA, APIs (Module 1)
9.1	ML Workflow: Problem Framing, Features vs Labels, Train/Val/Test Split, Baselines
9.2	Data Prep: Missing Values, Duplicates, Encoding, Scaling (on CarDekho used cars data)

The Story

You've cleaned the CarDekho dataset. It's ML-ready. You're about to train your first model. But before you do — there are **traps** that can make your model look amazing on paper but fail completely in the real world.

Today we learn about the three biggest traps in ML:

1. **Data Leakage** — Your model is cheating without you knowing
2. **Target Distribution Skew** — Your model is lazy and predicts “average” for everything
3. **Cross-Validation** — Why a single train-test split can mislead you

These aren't coding problems. They're **thinking problems**. And they catch even experienced data scientists.

Part 1: Data Leakage — The Silent Killer

What is Data Leakage?

Data leakage happens when information from **outside the training data** sneaks into your model during training. The model learns patterns it shouldn't have access to, and its performance looks unrealistically good.

Think of it like this:

Imagine you're preparing for a college exam. You study hard, take the exam, and score 95%. Impressive!

But what if someone accidentally gave you the answer key while you were studying? You'd still score 95% — but it means nothing. You didn't actually learn the material. You just memorized the answers.

Data leakage is giving your model the answer key during training.

The scary part? Your model's accuracy will look great. You'll think it's working. But the moment it sees truly new data (in production), it falls apart.

Why is Leakage Dangerous?

With Leakage	Without Leakage
Model accuracy: 95% ✨	Model accuracy: 72%
You celebrate and deploy	You investigate and improve
Model fails in production 🌟	Model works as expected ✅
Your manager is angry	Your manager is happy

The problem isn't that the model is bad — it's that **you don't know it's bad** until it's too late.

Types of Data Leakage

There are three main types. Let's go through each one.

Type 1: Train-Test Contamination

What: Information from the test set leaks into the training process.

The most common example: Scaling or encoding BEFORE splitting.

- ❌ WRONG (Leakage):
1. Scale ALL data (scaler learns mean/std from everything, including test)
 2. Split into train/test
 3. Train model
- The model has indirectly "seen" test data through the scaler's statistics
- ✅ CORRECT (No Leakage):
1. Split into train/test
 2. Fit scaler on TRAIN only
 3. Transform both train and test using train's statistics
- Test data is truly unseen

Real-life analogy:

You're a teacher making a test for your students. You write 100 questions, then pick 20 for the exam. But while grading practice papers, you accidentally used all 100 questions as examples. Some students saw the exact exam questions during practice. Their scores are inflated — they didn't learn better, they just got lucky.

How to prevent it:

- Always split FIRST, then preprocess
- Use `sklearn.pipeline.Pipeline` to ensure preprocessing happens inside cross-validation
- Never use test data statistics (mean, std, min, max) during training

Type 2: Target Leakage (Feature Leakage)

What: A feature in your training data contains information that is **derived from or strongly correlated with the target variable** — information that wouldn't be available at prediction time.

Example with our CarDekho data:

Imagine someone added a column called `price_category` to the dataset:

```
selling_price    price_category
120000           Budget
550000           Mid-Range
2500000          Premium
```

If you train a model to predict `selling_price` using `price_category` as a feature, it will get near-perfect accuracy. But that's cheating — `price_category` was CREATED from `selling_price`. At prediction time, you won't have this column because you don't know the price yet (that's what you're predicting!).

More subtle examples:

Feature	Target	Leakage?	Why
<code>price_category</code> (Budget/Mid/Premium)	<code>selling_price</code>	✅ Yes	Derived directly from target
<code>total_emi_paid</code> (total EMI payments made)	<code>selling_price</code>	✅ Yes	Only known after the sale
<code>days_listed</code> (how long the ad was up)	<code>selling_price</code>	⚠️ Maybe	Could be influenced by price — cheap cars sell fast
<code>km_driven</code>	<code>selling_price</code>	❌ No	Known before the sale, independent of price
<code>vehicle_age</code>	<code>selling_price</code>	❌ No	Known before the sale

The key question to ask for every feature:

"Would I have this information at the time I need to make the prediction?"

If the answer is no — it's leakage. Drop it.

Real-life analogy:

You're predicting whether a student will pass the final exam. One of your features is "final_grade". Well, of course the model will be 100% accurate — the final grade IS the answer! But you don't have the final grade before the exam happens. That feature wouldn't exist at prediction time.

Type 3: Temporal Leakage (Time-Based Leakage)

What: Using future data to predict the past. This happens with time-series data when you don't respect the time order.

Example:

You're predicting used car prices for January 2024. Your training data includes:

- Cars sold in 2022 ✅ (past — fine)
- Cars sold in 2023 ✅ (past — fine)
- Cars sold in February 2024 ❌ (future — leakage!)

If February 2024 data is in your training set, the model has seen "the future" and will perform unrealistically well on January 2024 predictions.

How to prevent it:

- Always split time-series data **chronologically** (train on past, test on future)
- Never shuffle time-series data randomly before splitting
- Use `TimeSeriesSplit` from sklearn instead of regular `train_test_split`

```
❌ WRONG: Random split on time-series data
Train: [Jan, Mar, May, Jul, Sep, Nov] ← future months mixed in!
Test:  [Feb, Apr, Jun, Aug, Oct, Dec]

✅ CORRECT: Chronological split
Train: [Jan, Feb, Mar, Apr, May, Jun, Jul, Aug]
Test:  [Sep, Oct, Nov, Dec] ← only future months
```

The Data Leakage Checklist

Use this checklist before training ANY model:

- [] Did I split the data BEFORE any preprocessing (scaling, encoding, imputation)?
- [] Are all my features available at prediction time? (No future information)
- [] Is any feature derived from or strongly correlated with the target?

- [] If time-series: did I split chronologically (not randomly)?
- [] Am I using `fit_transform` only on training data and `transform` on test?
- [] If using cross-validation: is preprocessing inside the CV loop (via Pipeline)?

⚠ Common Mistakes — Data Leakage

1. Scaling all data before splitting — the scaler “sees” test data statistics
2. Using `df.describe()` on the full dataset to decide imputation values, then splitting — the imputation values contain test data info
3. Feature engineering using the target column (e.g., creating “price_per_km” when predicting price)
4. Not checking if a feature is “from the future” — always ask “would I have this at prediction time?”
5. Shuffling time-series data before splitting

🔗 Demo: See Leakage in Action

Run the script `9.3_leakage_demo.py` to see how data leakage inflates model performance. The script trains the same model twice — once WITH leakage and once WITHOUT — and shows the difference in evaluation metrics.

You don’t need to understand the model code yet. Just look at the numbers.

Part 2: Target Distribution Skew (The Regression Version of Class Imbalance)

Class Imbalance — The Classification Problem

In classification (predicting categories like “spam” or “not spam”), **class imbalance** means one class has way more examples than the other:

```
Email Dataset:
  Not Spam: 9,500 emails (95%)
    Spam:      500 emails (5%)
```

A lazy model can just predict “Not Spam” for everything and get 95% accuracy. But it catches zero spam — completely useless.

📌 **Note:** We’ll deal with class imbalance hands-on when we get to classification models. For now, understand the concept — it’ll click when we get there.

Target Distribution Skew — The Regression Problem

In regression (predicting numbers like car prices), the equivalent problem is **skewed target distribution**:

CarDekho Selling Prices:

₹1-5 lakh: ~60% of cars ← most cars are here
₹5-15 lakh: ~30% of cars
₹15-40 lakh: ~8% of cars
₹40+ lakh: ~2% of cars ← very few luxury cars

A lazy model can predict “₹5 lakh” for everything and have a decent average error. But it will be terrible at predicting luxury car prices — and those are often the most important predictions for the business.

Why this matters:

- The model sees 100x more budget cars than luxury cars during training
- It learns budget car patterns really well, luxury car patterns poorly
- Average metrics (like Mean Absolute Error) look okay because they're dominated by the majority
- But per-segment performance is terrible for rare price ranges

How to Detect Target Skew

```
# Check the distribution
df['selling_price'].describe()
df['selling_price'].hist(bins=50)

# Check skewness
df['selling_price'].skew() # > 1 means heavily right-skewed
```

How to Handle Target Skew in Regression

Approach	What It Does	When to Use
Log Transform the target	<code>y = np.log1p(selling_price)</code> — compresses the range	When target is right-skewed (most prices are low, few are very high)
Stratified Sampling	Ensure train/test have similar price distributions	Always — use bins to stratify
Weighted Loss	Tell the model to care more about rare price ranges	When rare segments are business-critical
Separate Models	Train one model for budget cars, another for luxury	When segments behave very differently

💡 **Key Idea:** Log-transforming the target is the simplest and most common fix for skewed regression targets. Instead of predicting ₹5,00,000, the model predicts $\log(5,00,000) \approx 13.1$. This compresses the range and makes the distribution more symmetric. You transform back with `np.expml()` after prediction.

SMOTE — Synthetic Data for Imbalanced Classification

SMOTE (Synthetic Minority Over-sampling Technique) is a technique that creates **synthetic (fake but realistic) samples** of the minority class to balance the dataset.

Before SMOTE:	After SMOTE:
Not Spam: 9,500	Not Spam: 9,500
Spam: 500	Spam: 9,500 (500 real + 9,000 synthetic)

How SMOTE works (simplified):

1. Pick a minority class sample
2. Find its K nearest neighbors (also minority class)
3. Create a new synthetic sample somewhere between the original and a random neighbor
4. Repeat until classes are balanced

Original points:	A •	• B	(both are "Spam")
SMOTE creates:	A •	●	• B (● is a synthetic point between A and B)

For regression, SMOTE doesn't directly apply (there are no "classes" to balance). The regression equivalents are:

- **SMOEN** — Synthetic Minority Over-sampling for Regression (creates synthetic samples in rare target ranges)
- **Oversampling** rare price ranges
- **Log transform** (simpler and usually sufficient)

 **Note:** We'll use SMOTE hands-on when we reach classification models. For now, understand the concept: it creates fake-but-realistic data points to balance an imbalanced dataset. For our regression problem, log-transforming the target is the practical solution.

⚠ Common Mistakes — SMOTE

1. Applying SMOTE BEFORE splitting — synthetic data leaks into the test set. Always apply SMOTE only on the training set.
2. Using SMOTE when you don't need it — if classes are 60/40, that's not really imbalanced. SMOTE is for severe imbalance (95/5 or worse).
3. Applying SMOTE to regression problems — SMOTE is for classification. Use log transform or SMOEN for regression.

Part 3: Cross-Validation — Don't Trust a Single Split

The Problem with a Single Train-Test Split

In session 9.1, we learned to split data into train and test sets. But here's the problem:

Your model's performance depends on **which data** ends up in train vs test.

Imagine you have 100 students. You randomly pick 80 for training and 20 for testing. What if, by pure luck:

- All the easy-to-predict students ended up in the test set → accuracy looks great
- All the hard-to-predict students ended up in the test set → accuracy looks terrible

Same model, same data, different split → different results. That's not reliable.

```
Split 1: Train = [easy cars]    Test = [easy cars]    R² = 0.92 🌟
Split 2: Train = [mixed cars]   Test = [mixed cars]   R² = 0.85
Split 3: Train = [easy cars]    Test = [hard cars]    R² = 0.71 🤔

Same model, same data. Three different answers. Which one is "real"?
```

The Solution: K-Fold Cross-Validation

Instead of one split, do **K splits** and average the results.

How K-Fold works (K=5 example):

```
Step 1: Divide data into 5 equal parts (folds)

All Data: [████████████████████████████████████████]
           [Fold 1][Fold 2][Fold 3][Fold 4][Fold 5]

Step 2: Train 5 times, each time using a different fold as the test set

Round 1: [TEST ][Train][Train][Train][Train] Score: 0.83
Round 2: [Train][TEST ][Train][Train][Train] Score: 0.87
Round 3: [Train][Train][TEST ][Train][Train] Score: 0.81
Round 4: [Train][Train][Train][TEST ][Train] Score: 0.85
Round 5: [Train][Train][Train][Train][TEST ] Score: 0.84

Step 3: Average all scores
Final Score: (0.83 + 0.87 + 0.81 + 0.85 + 0.84) / 5 = 0.84 ± 0.02
```

Why this is better:

- Every data point gets to be in the test set exactly once
- You get 5 scores instead of 1 — you can see the variance
- The average is more reliable than any single split
- The standard deviation tells you how stable your model is

Choosing K (Number of Folds)

K Value	Pros	Cons	When to Use
K=5	Good balance of speed and reliability	Moderate variance	Default choice — works for most cases
K=10	More reliable estimate	Slower (10 training rounds)	When you want higher confidence
K=3	Fast	Higher variance	Large datasets (100K+ rows) where training is slow
K=N (Leave-One-Out)	Uses maximum training data	Very slow, high variance	Tiny datasets (< 100 rows)

💡 **Key Idea:** *K=5 is the industry default. Use K=10 if you want to be extra careful. Don't overthink this — the exact K matters less than actually doing cross-validation instead of a single split.*

Stratified K-Fold

Regular K-Fold splits data randomly. But what if one fold accidentally gets all the luxury cars and another gets all the budget cars? The scores would be misleading.

Stratified K-Fold ensures each fold has a similar distribution of the target variable.

```
Regular K-Fold (risky):  
Fold 1: [mostly budget cars]   Score: 0.90 (easy fold)  
Fold 2: [mostly luxury cars]  Score: 0.65 (hard fold)  
❑ High variance between folds  
  
Stratified K-Fold (better):  
Fold 1: [mix of budget + luxury] Score: 0.84  
Fold 2: [mix of budget + luxury] Score: 0.83  
❑ Consistent scores across folds
```

For classification, `StratifiedKFold` preserves class proportions. For regression, you can bin the target into ranges and stratify on those bins.

Cross-Validation + Preprocessing: Avoiding Leakage Inside CV

Here's a subtle but critical point: if you preprocess (scale, encode) BEFORE cross-validation, you have leakage. The scaler saw all the data, including the fold that's being used as the test set.

- ❌ **WRONG:**
1. Scale all data
 2. Run 5-fold CV
- Each "test fold" was already seen by the scaler
- ✅ **CORRECT:**
1. Use sklearn Pipeline
 2. Pipeline does: scale → train → predict **INSIDE** each fold
- Each fold's test data is truly unseen

This is why `sklearn.pipeline.Pipeline` exists — it ensures preprocessing happens fresh inside each fold.

When to Use Cross-Validation

Situation	Use CV?	Why
Comparing two models	✅ Yes	Single split might favor one model by luck
Tuning hyperparameters	✅ Yes	Need reliable scores to compare settings
Final model evaluation	✅ Yes	Get a confidence interval, not just a point estimate
Very large dataset (1M+ rows)	⚠️ Maybe	A single 80/20 split is usually stable enough; CV is slow
Time-series data	⚠️ Special	Use <code>TimeSeriesSplit</code> , not regular K-Fold

Shortcomings of Cross-Validation

Cross-validation is great, but it's not perfect:

1. **Slower** — K=5 means training 5 models instead of 1. For large datasets or complex models, this can be very slow.
2. **Doesn't give you a final model** — CV gives you a score, not a trained model. After CV, you still need to train a final model on all training data.
3. **Assumes data is i.i.d.** — If your data has time dependencies or groups (e.g., multiple listings from the same seller), regular K-Fold can leak information between folds.
4. **Can still be misleading** — If your entire dataset is biased (e.g., only cars from Bangalore), CV won't tell you the model will fail on Delhi cars.

⚠️ Common Mistakes — Cross-Validation

1. *Preprocessing outside the CV loop — this is data leakage. Use Pipeline.*
2. *Using regular K-Fold on time-series data — use TimeSeriesSplit instead.*
3. *Reporting only the mean CV score without the standard deviation — “0.85 ± 0.02” is much more informative than just “0.85”.*
4. *Using CV on the test set — CV is for the training set. The test set is held out completely and used only once at the very end.*

5. Thinking CV replaces a test set — it doesn't. CV helps you choose the best model/settings. The test set gives you the final unbiased evaluation.

Summary

Concept	What It Is	Key Takeaway
Data Leakage	Model sees information it shouldn't during training	Always split BEFORE preprocessing; ask "would I have this at prediction time?"
Target Skew	Most data is in one range, model ignores rare ranges	Log-transform the target for regression; SMOTE for classification (later)
Cross-Validation	Train K times with different splits, average the scores	More reliable than a single split; use K=5 as default
SMOTE	Creates synthetic minority samples for classification	Apply only on training data; not for regression (use log transform instead)

Next session (9.4): We'll train our first regression model on the clean CarDekho data and evaluate it properly using everything we learned today.

Trust your validation. Question your metrics. And never, ever scale before splitting. 🛡️